

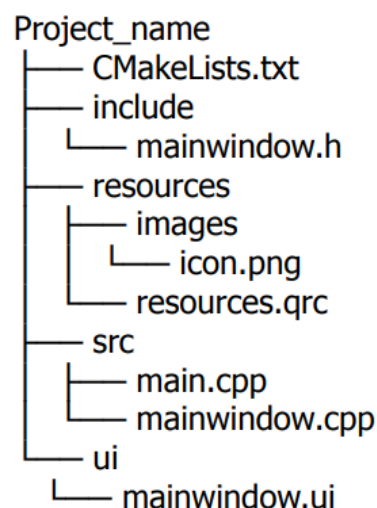
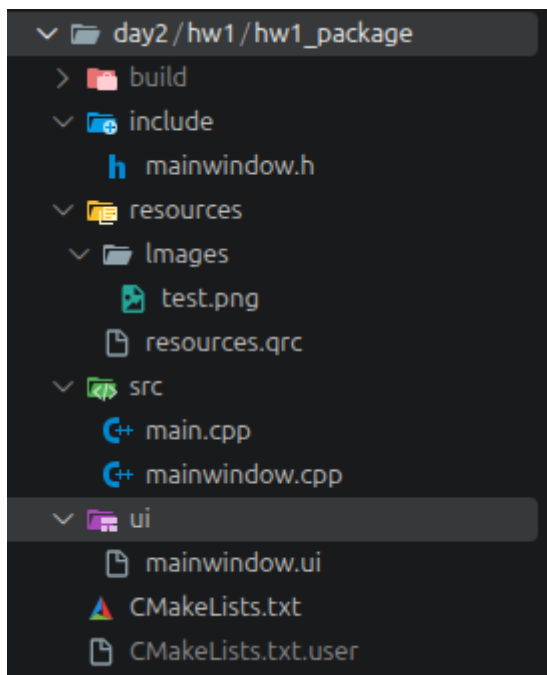
# CQt day2 hw1 레포트

로봇 예비단원 주호진

## 1. 과제

- 간단한 로봇 컨트롤 패널 UI 제작
- QPushButton을 활용해 로봇 이동 방향 제어할 수 있도록 구현
- QLabel로 현재 로봇의 상태 표시
- QSlider로 로봇의 속도 조절, 현재 속도 값을 UI에 표시
- 버튼 입력에 따라 상태 변경이 가능하도록 Signal / Slot 기능 활용
- UI 구성은 자유롭게 제작, 전진 / 후진 / 좌회전 / 우회전 / 정지 기능 필수
- 실제 로봇과는 연결할 필요는 X 버튼 입력에 따른 UI 동작만 간단하게 구현

## 2. 폴더 구조

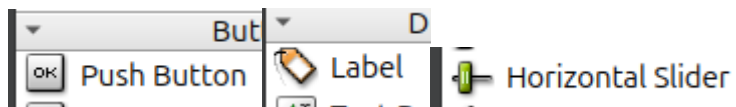


위와 같은 구조로 설정

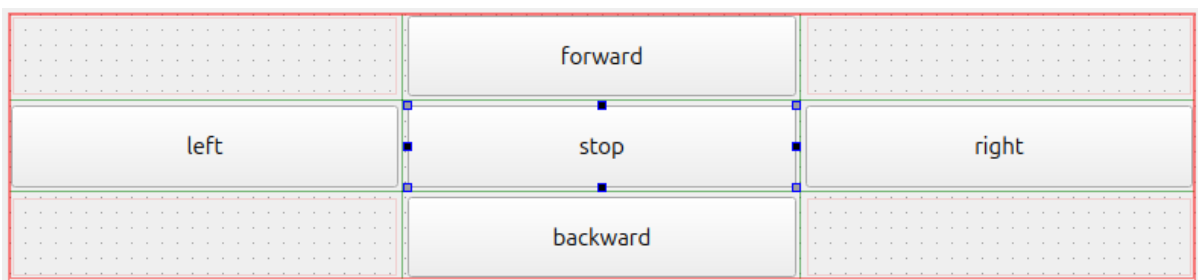
폴더 구조는 ppt에 제시된 구조대로 설정하였고 .gitignore을 통해 build 폴더와 CMakeLists.txt.user 파일은 변경 사항에 적용되지 않도록 했다.

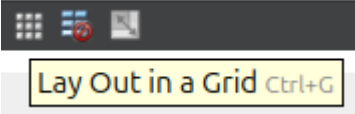
### 3. 코드 설명

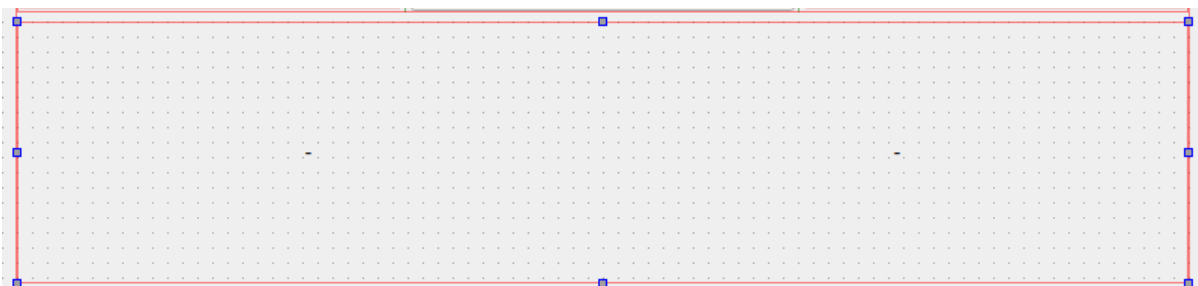
#### 3-1. Mainwindow.ui



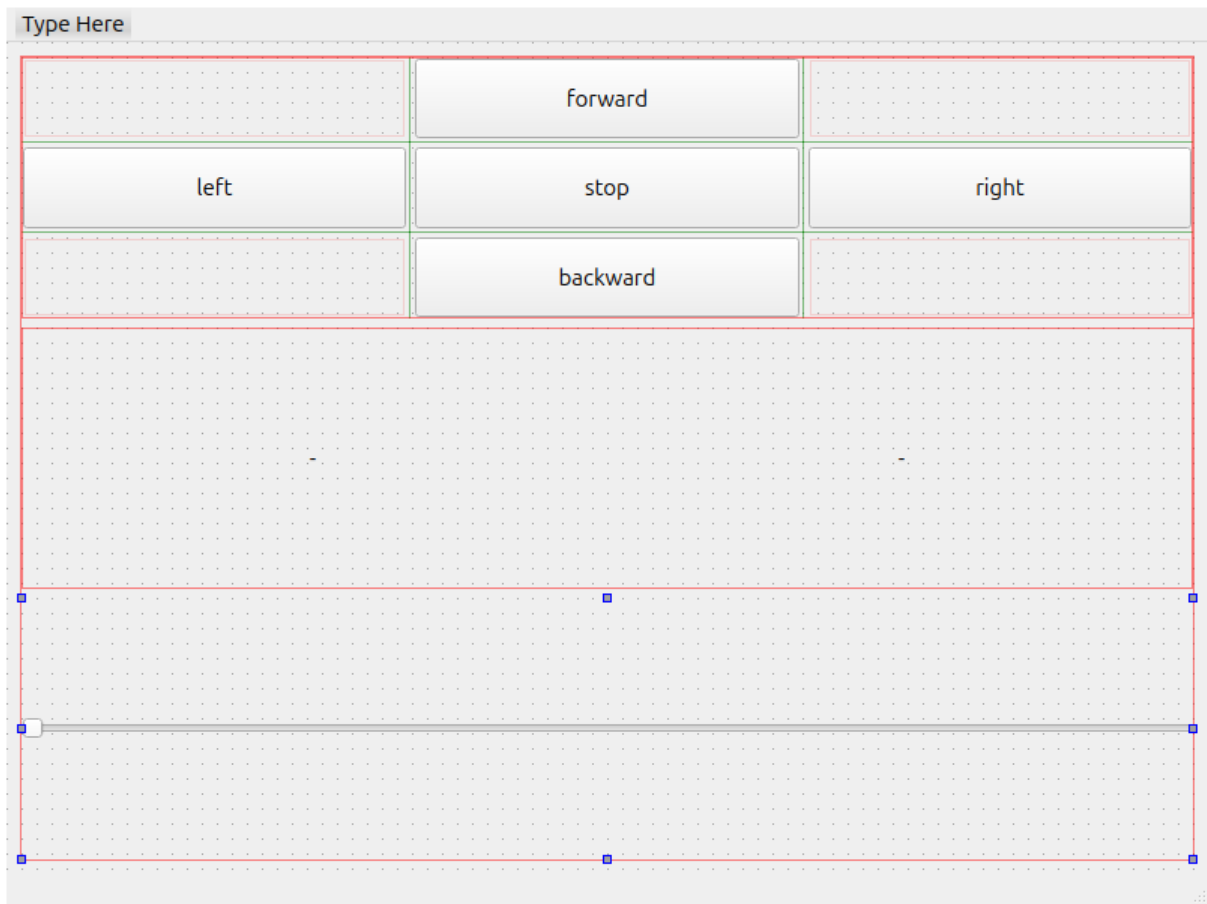
위 3개의 위젯을 사용하였다.



버튼은  lay out grid 버튼을 사용하여 위와 같이 배치해주었다.



Label은 horizon으로 묶어서 2개를 나란히 배치했다.



위 2개의 위젯과 horizon slider를 vertical로 배치하여 위 사진처럼 ui를 설정하였다.

| Property Editor        |               |
|------------------------|---------------|
| Filter                 |               |
| speed_slider : QSlider |               |
| Property               | Value         |
| Country                | United States |
| inputMethodHints       | ImhNone       |
| QAbstractSlider        |               |
| minimum                | 0             |
| <b>maximum</b>         | 10            |
| singleStep             | 1             |
| pageStep               | 10            |
| value                  | 0             |

Slider의 범위는 property Editor에서 maximum값을 수정해주었다.

### 3-2. Mainwindow.h

```
private:
    QString current_state; // forward, backward, left, right, stop
    int speed;
```

헤더파일에는 현재 방향을 표시해줄 state와 slider값을 저장할 speed를 선언해주었다. 처음에 current\_state를 char 배열로 선언하였는데 qt에서 char 배열이 수정이 안되어서 QString으로 변경하였다.

```
private slots:
    void on_forward_pushBtn_clicked();

    void on_backward_pushBtn_clicked();

    void on_left_pushBtn_clicked();

    void on_right_pushBtn_clicked();

    void on_stop_pushBtn_clicked();

    void on_speed_slider_valueChanged(int value);
```

추가적으로 .ui 파일에서 추가한 slots이 있다.

### 3-3. Mainwindow.cpp

```
void MainWindow::on_forward_pushBtn_clicked()
{
    current_state = "forward";
    ui -> status_label -> setText(current_state);
}
```

헤더파일에서 선언한 slot의 기능을 명시해주었다. 각각 버튼에 맞는 state로 변경되도록 했고, statuslabel(ui상 왼쪽 label)에 state를 띄우도록 하였다. Backward, left, right, stop 모두 위 코드에서 current\_state만 다르고 코드 구조는 동일하다.

```
void MainWindow::on_speed_slider_valueChanged(int value)
{
    speed = value;
    QString temp = QString::number(speed);
    ui -> speed_label -> setText(temp);
}
```

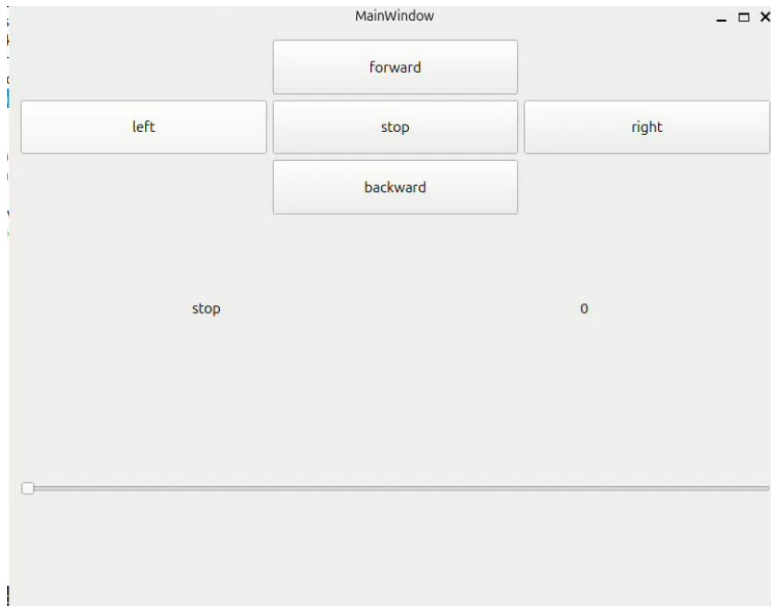
슬라이더를 속도와 연결해주어야 하기 때문에 슬라이더의 값이 변하는 signal을 이용했다. Value를 speed에 저장하여 준 뒤에, setText에 표시하기 위해서 QString 자료형으로 변환하여 넣어주었다.

## 4. 실행 결과 화면

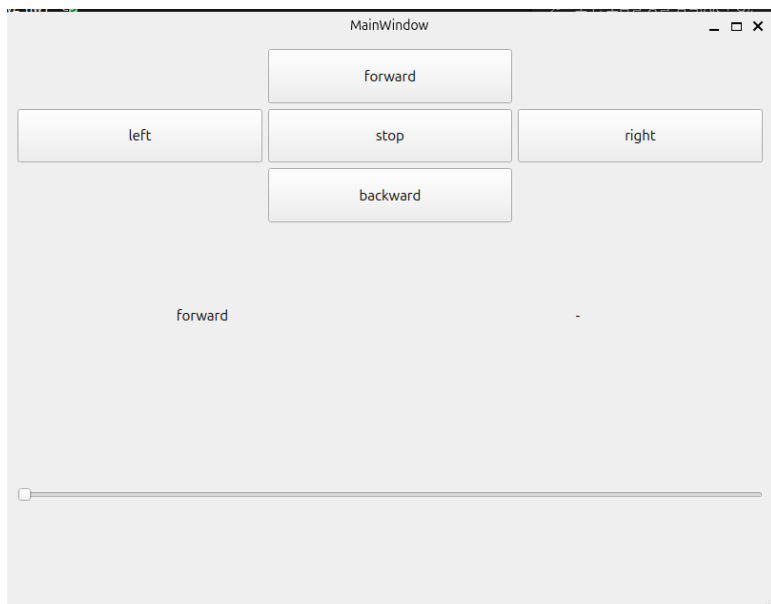
## 실행 영상

### 캡처 화면

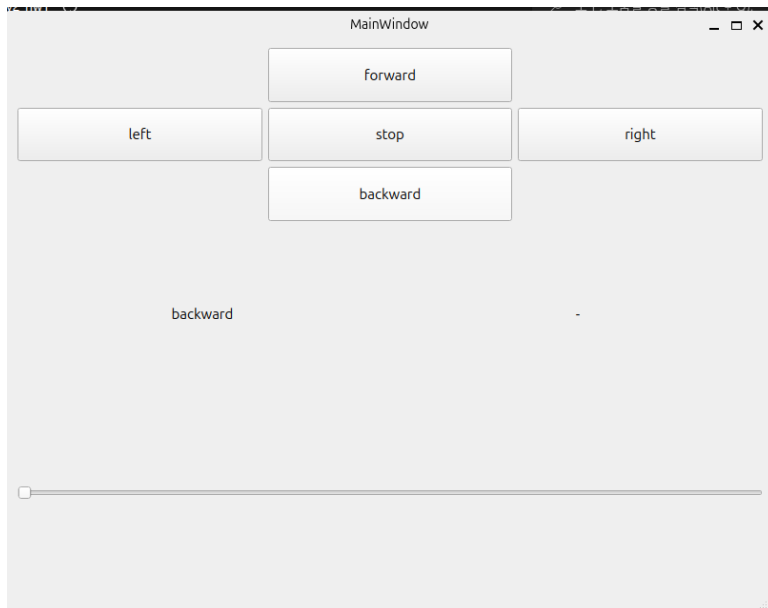
#### 4-1. Stop 눌렀을 때



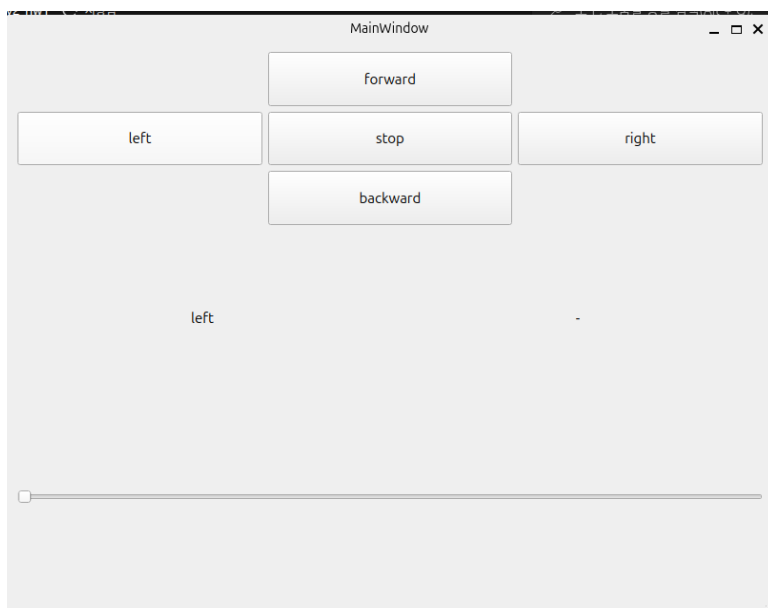
#### 4-2. Forward 눌렀을 때



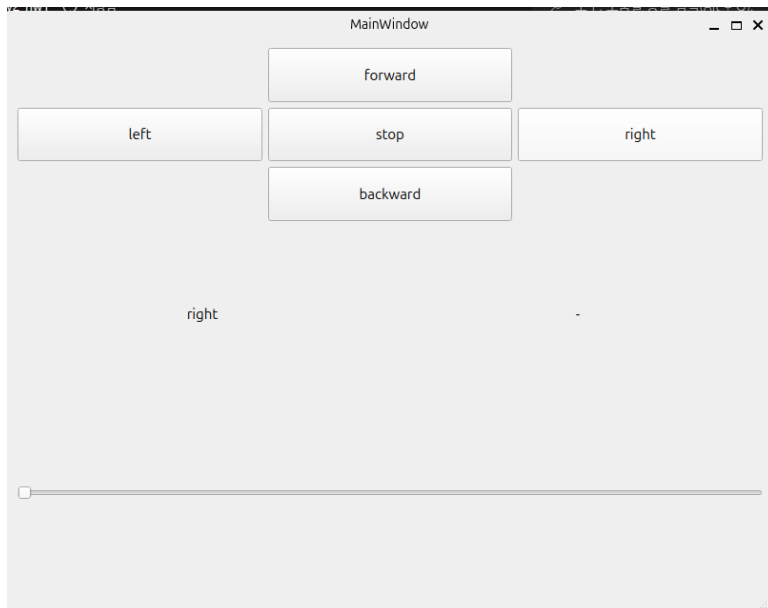
#### 4-3. Backward 눌렀을 때



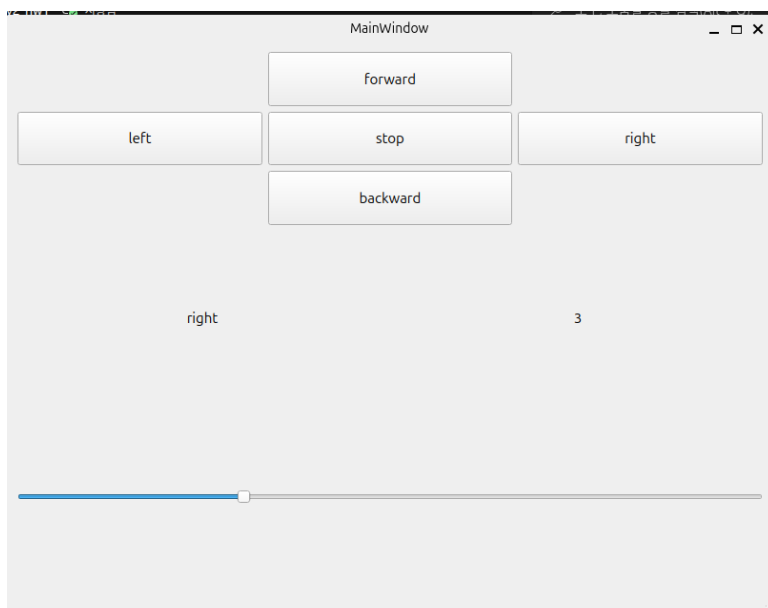
4-4. left 눌렀을 때

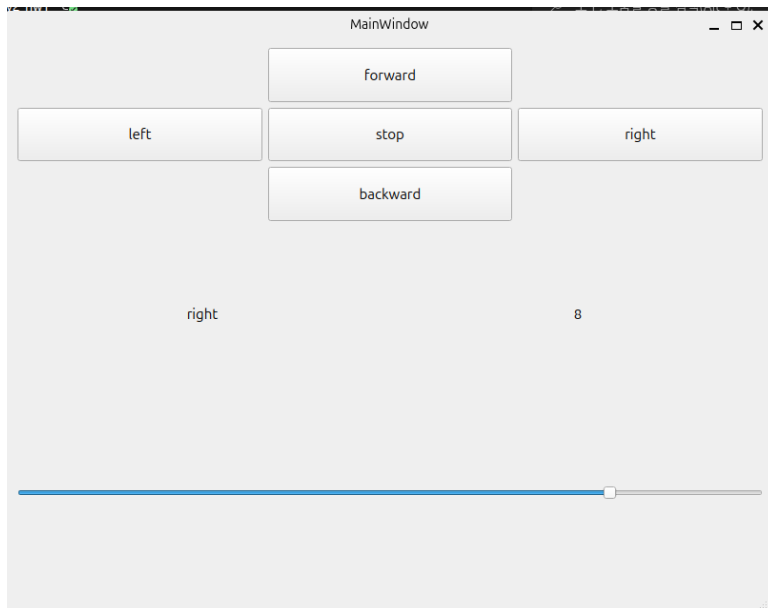


4-5. right 눌렀을 때



#### 4-6. 슬라이더 움직일 때(값이 변할 때)





## 5. AI 사용 보고서

<https://share.gemini.google/Nlx2tkoEzSVV>

답변이 긴 질문은 질문 캡처본만 올려서 정리하였다.

qt slot signal connect 개념 정리 및 예제도 같이

Qt의 시그널과 슬롯(Signal & Slot)은 객체 간의 통신을 위해 사용되는 Qt 고유의 핵심 메커니즘으로, 특정 이벤트가 발생했을 때 다른 객체에 안전하고 유연하게 알림을 전달합니다.

push 버튼과 qlabel 예제

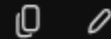
qt qlabel signal 설명좀 해줘



Qt에서 사용하는 개념 및 예제를 요청했다. Slot, signal, connect 내용에 대해 정리, QLabel에 있는 signal을 설명해달라고 하였다. 개념이 잘 이해가 안되어서 비유 표현을 통해 이해하였다. 그리고 코드 적용을 위해 예제를 요청했으나 예제를 보고 이해가 되지 않아서 아래에 추가 질문을 하였다.

```
void MainWindow::on_pushButton_clicked(){  
    ui->lineEdit->setText("Hello World!");}
```

블로그 예제가 이렇게 되어 있던데 ui->lineEdit->setText("Hello World!"); 이부분 설명좀



AI로 받은 예제가 이해가 안돼서 구글링을 하다가 한 블로그에서 예제 코드를 긁어서 질문하였다. 해당 질문에 QLineEdit이 뭔지 몰라서 질문했었는데, .ui파일에서 설정한 위젯의 이름을 설정하는 것이었다.

다른 위젯의 slot으로 QLabel에 값을 표시하려면 어떻게 해야 돼

signal과 slot을 connect로 연결하라는데

```
void MainWindow::on_stop_pushBtn_clicked()
{
    current_state = "stop";
    ui->status_label->setText(current_state);
}

void MainWindow::on_speed_slider_valueChanged(int value)
{
    speed = value;
    QString temp = QString::number(speed);
    ui->speed_label->setText(temp);
}
```

그냥 이런식으로 작성해도 작동되는데 뭐가 빠진 게 있나



그 코드가 정상적으로 작동하는 이유는 Qt가 제공하는 **자동 시그널 연결(Automatic Connection)** 기능 덕분입니다. 빠진 것은 없으며, Qt가 내부적으로 처리해 주는 정상적인 방식입니다.

qt에서 사용하는 코드 구조에 대해 질문하였다. 개념에서 connect라는 게 있고 connect로 이어주어야 작동한다고 보았는데, 컴파일 해봤을 때 문제 없이 작동하여서 질문하였다. Qt 상에서 자동 시그널 연결이라는 기능을 통해 특정 문법을 사용하면 connect를 자동으로 생성해준다고 한다.

setText에 int형은 못넣어?

네, `setText()` 는 문자열( `QString` )만 받기 때문에 `int` 형을 직접 넣을 수 없습니다.  
`QString::number()` 등을 사용해 문자열로 변환한 뒤 전달해야 합니다.

.ui파일에서 추가하거나 수정한 부분은 코드상으로 볼 수 없어?

네, 빌드 과정에서 컴파일러( `uic` )가 `.ui` 파일을 C++ 헤더 코드(예: `ui_mainwindow.h` )로 자동 변환하므로 빌드된 폴더나 IDE의 생성 파일 목록에서 코드로 확인할 수 있습니다.

인터넷 상의 예제를 보면 대부분이 .ui에서 추가하지 않고 코드상에서 작성해서 띄워 사용했는데, 내 프로젝트에는 동일한 위젯인데도 코드가 없어서 질문을 했다. .ui 파일을 통해 추가한 위젯은 컴파일할 때 자동으로 읽어오고, 코드로는 따로 추가되지 않는다고 하였다. 오늘 qt를 처음 접해서 .ui 파일로 위젯을 끌어다가 썼는데 앞으로는 코드 상에서 구현을 해야겠다고 생각했다.

+ CMakeLists.txt 설정

<https://share.gemini.google/9CbdVhviN0lp>

ppt에 있는 CMakeLists.txt를 설정한 뒤에 컴파일이 되지 않아서 CMakeList.txt를 수정하는 데에 LLM을 사용했다. (아래의 4줄을 추가하였다.)

```
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTOUIC ON)
set(CMAKE_AUTORCC ON)
set(CMAKE_AUTOUIC_SEARCH_PATHS "${CMAKE_CURRENT_SOURCE_DIR}/ui")
```